1. @SpringBootApplication : is added on the main class , in spring boot application , to run it as a standalone JAR / WAR

@SpringBootApplication =

1.1 @Configuration – Designates this main class as a configuration class for Java configuration. In addition to beans configured via component scanning, an application may want to configure some additional beans via the @Bean annotation as demonstrated here. Thus, the return value of methods having the @Bean annotation in this class are registered as beans.

1.2 @EnableAutoConfiguration – This enables Spring Boot's autoconfiguration mechanism. Auto-configuration refers to creating beans automatically by scanning the classpath.

1.3 @ComponentScan – Typically, in a Spring application, annotations like @Component, @Configuration, @Service, @Repository are specified on classes to mark them as Spring beans. The @ComponentScan annotation basically tells Spring Boot to scan the current package and its sub-packages in order to identify annotated classes and configure them as Spring beans. Thus, it treats the current package as the root package for component scanning.

Annotation added automatically :
@EnableWebMvc: Marks the application as a web application and activates setting up a DispatcherServlet , HandlerMapping , ViewResolvers . Spring Boot adds it automatically when it sees spring-webmvc on the classpath.

2. How to use the @SpringBootApplication annotation
In order to run a Spring Boot application, it needs to have a class with the @SpringBootApplication annotation.

eg :
```
package com.app;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class Main {
    public static void main(String[] args) {
          SpringApplication.run(Main.class, args);
      }
}
```
The Main class has the @SpringBootApplication annotation

It simply invokes the SpringApplication.run method.
This starts the Spring application as a standalone application, runs the embedded servers and loads the beans.

Normally, such a main class is placed in a root package above other packages. This enables component scanning to scan all the sub-packages for beans.